

Relazione Homework 3

Gruppo di lavoro:

Roberta Vallelunga (Matricola: 1000003065)

Antonio Zarbo (Matricola: 1000081002)

Simone Squillaci (Matricola: 1000081013)

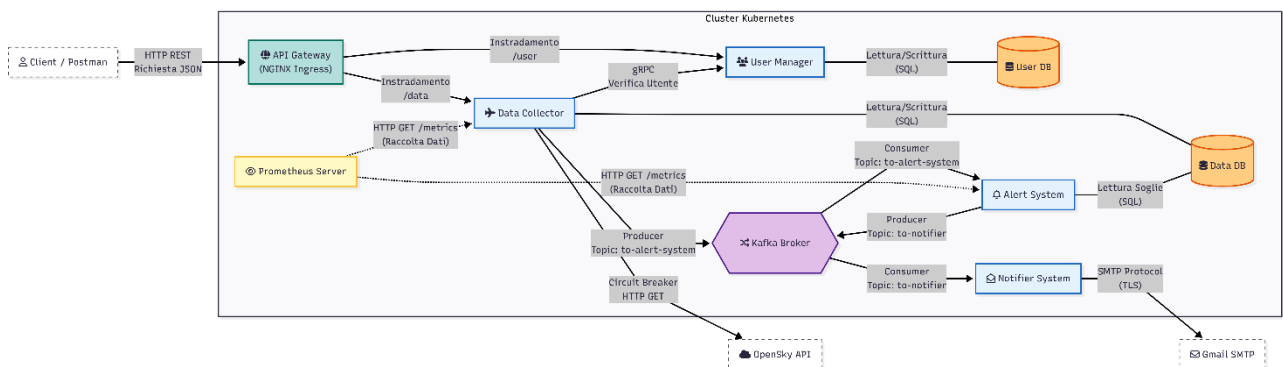
1. Descrizione del progetto

L'obiettivo del terzo homework è l'evoluzione del sistema a microservizi attraverso la sua migrazione verso un'infrastruttura di orchestrazione professionale e l'implementazione di strategie di monitoraggio avanzate. Il sistema, pur mantenendo le logiche di business consolidate (gestione utenti gRPC, resilienza con Circuit Breaker e notifiche asincrone via Kafka), è stato esteso introducendo:

- **Orchestrazione con Kubernetes (K8s):** Migrazione da Docker Compose a un cluster Kubernetes (distribuzione *kind*).
- **White-box Monitoring:** Implementazione di metriche interne nei microservizi critici utilizzando Prometheus.
- **Sicurezza e Isolamento:** Definizione di Network Policies per il controllo granulare del traffico tra i pod.

2. Architettura

Il sistema abbandona la gestione a container singoli per adottare un'architettura a **Pod** orchestrati. La comunicazione tra i servizi avviene ora tramite i **Service** di Kubernetes.



Schema architetturale del sistema

Per garantire la durabilità dei dati in un ambiente effimero come Kubernetes, è stata implementata una strategia di persistenza basata su PersistentVolume (PV) e PersistentVolumeClaim (PVC).

2.1 Componenti Principali

Prometheus è un sistema open-source per il monitoraggio e la raccolta di metriche progettato per ambienti distribuiti. Prometheus è stato distribuito come un Pod dedicato all'interno del cluster. Esso è configurato per effettuare lo scraping periodico dei dati dagli altri microservizi sfruttando il DNS interno di Kubernetes: i target nel file di configurazione (prometheus-k8s.yaml) puntano direttamente ai nomi dei Service Kubernetes, permettendo al sistema di monitoraggio di raggiungere i pod corretti.

I seguenti microservizi sono stati aggiornati per esporre un endpoint /metrics sulla porta 8000, permettendo a Prometheus di effettuare lo scraping delle performance:

DataCollector: Oltre alle funzioni precedenti, integra il monitoraggio dei tempi di risposta delle API esterne. Nello specifico:

- `datacollector_requests_total` (Counter): Conta le richieste HTTP ricevute.
- `datacollector_opensky_fetch_seconds` (Gauge): Misura la latenza dell'ultimo download da OpenSky.

AlertSystem: Oltre alle funzioni precedenti, monitora l'efficienza della catena di notifica. Nello specifico:

- `alertsystem_sent_total` (Counter): Numero totale di alert inviati verso il topic Kafka.
- `alertsystem_message_processing_seconds` (Gauge): Tempo di elaborazione del singolo messaggio.

Ogni metrica è associata a due **label** fondamentali: `service`, per identificare il microservizio, e `node`, per tracciare il nodo del cluster su cui il pod è in esecuzione.

L'accesso dall'esterno è ora gestito da un **Ingress Controller** che instrada il traffico verso i Service del DataCollector e dello UserManager.

3. Scelte progettuali

L'architettura del sistema evolve verso un modello a **Microservizi orchestrati**, migrando dall'ambiente Docker Compose a un cluster **Kubernetes**.

Il passaggio all'orchestrazione professionale è stato gestito con un approccio incrementale. Per facilitare la migrazione, si è scelto inizialmente di consolidare tutte le componenti dell'architettura utilizzando **Docker**. Questo ha permesso di testare i singoli servizi e la logica

di business in un ambiente isolato ma semplificato, garantendo il pieno funzionamento di gRPC, Kafka e dei database prima del passaggio al cluster definitivo.

3.1 Migrazione a Kubernetes

In fase di migrazione, la configurazione imperativa di Docker Compose è stata tradotta in una configurazione dichiarativa tramite **manifest YAML**. In questi file sono stati descritti in dettaglio Pod, Service, Deployment, Persistent Volumes e Network Policies. Questo set di configurazioni viene processato da **Kind**, che orchestra le immagini Docker precedentemente create per istanziare i Pod e i relativi volumi all'interno dei nodi del cluster locale, ricreando l'intera topologia di rete richiesta.

Kubernetes adotta nativamente un modello di rete **flat**, in cui tutti i Pod possono comunicare liberamente tra loro. Per mantenere un adeguato livello di isolamento tra i servizi, analogo a quello ottenuto con Docker Compose, è stato necessario definire delle **NetworkPolicies**. Questa specifica stabilisce quali componenti possono comunicare tra loro e delimita le diverse “reti logiche” del sistema, garantendo che ogni servizio rimanga indipendente e accessibile solo dove previsto.

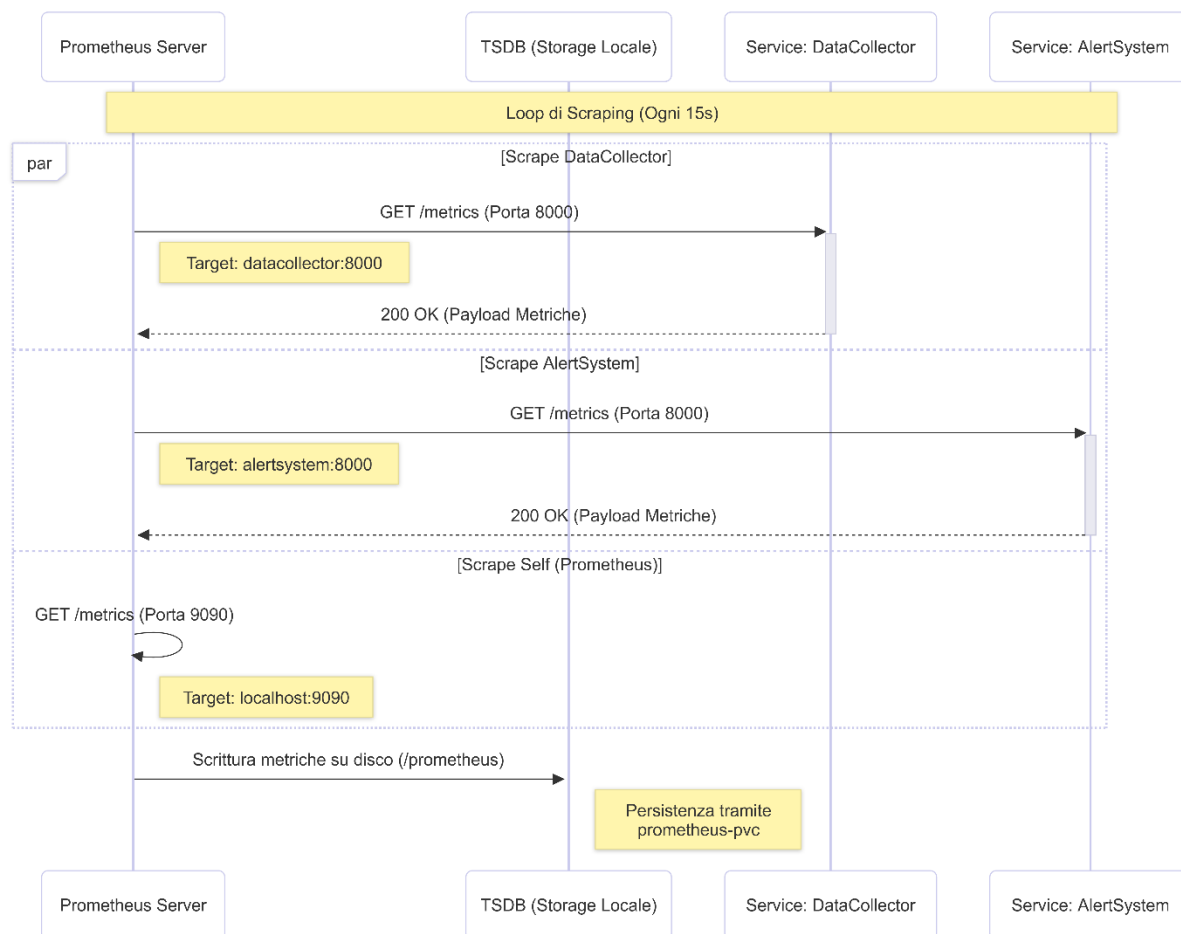
3.2 API Gateway

Per mantenere il ruolo di API Gateway nella transizione a Kubernetes, si è scelto di utilizzare l'**Ingress Controller**, in quanto utilizza come motore interno **NGINX**, già impiegato nella precedente architettura Docker Compose come punto di accesso unico al sistema. L'adozione di un Ingress Controller basato su NGINX consente di mantenere la stessa logica di routing, semplificando la transizione e garantendo che il punto di accesso unico al sistema mantenesse un comportamento coerente

3.3 Prometheus

Prometheus è stato introdotto prima della migrazione a Kubernetes, in modo da predisporre tutte le componenti necessarie al monitoraggio e definire le configurazioni iniziali all'interno del file *prometheus.yml*. Successivamente, per l'ambiente Kubernetes, la configurazione è stata estesa nel file *prometheus_k8s.yaml* (sotto forma di *ConfigMap*), adattando i target di scraping ai nomi dei Service del cluster. L'uso combinato di metriche e label (iniettate tramite variabili d'ambiente nei Pod) ha permesso di ottenere un'osservabilità granulare, distinguendo i dati non solo per microservizio ma anche per nodo fisico di esecuzione.

4. Diagrammi



Il diagramma di sequenza illustra il meccanismo di White-box monitoring basato su Prometheus. A intervalli regolari (configurati a 15 secondi), il server Prometheus avvia un ciclo di *scraping* per raccogliere i dati dai microservizi target.

Prometheus risolve gli indirizzi dei microservizi (`datacollector` e `alertsystem`) sfruttando il DNS interno del cluster Kubernetes. Le richieste HTTP `GET /metrics` vengono indirizzate alla porta **8000**, esposta specificamente dai pod per il monitoraggio, mantenendo separato il traffico di gestione da quello operativo (porta 5000). Il server monitora anche il proprio stato di salute interrogando l'endpoint `localhost:9090`.

Al termine del ciclo di raccolta, i dati vengono scritti sul *Time Series Database (TSDB)*, archivio locale delle metriche. Il diagramma mostra esplicitamente che tale storage è supportato da un **PersistentVolumeClaim (prometheus-pvc)**, garantendo che lo storico delle metriche sopravviva a eventuali riavvii o ricollocamenti del pod Prometheus.

5. Dettagli implementativi

L'interfaccia REST del sistema è accessibile dall'esterno esclusivamente tramite l'**Ingress Controller**. Di seguito è riportata la mappatura delle rotte configurate nel file ingress.yaml, che indirizzano il traffico verso i corretti Service Kubernetes:

Metodo	Endpoint (Path)	Servizio di Backend	Descrizione
POST	/new	userManager-service	Registrazione nuovi utenti (con politica At-Most-Once).
POST	/add	datacollector-service	Aggiunta interesse e soglie di monitoraggio.
POST	/modify_pref	datacollector-service	Modifica delle soglie per un aeroporto esistente.

Nota: I microservizi espongono anche un endpoint GET /metrics sulla porta interna 8000, che non è mappato sull'Ingress in quanto riservato esclusivamente allo scraping del server Prometheus all'interno della rete del cluster.